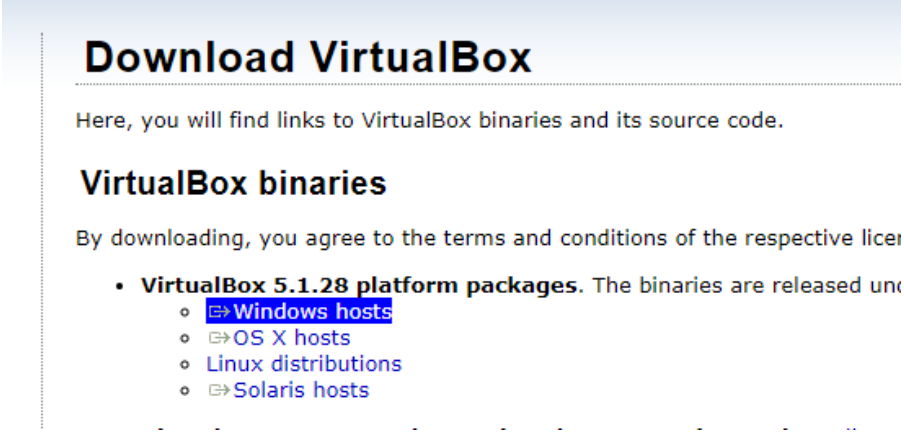
Guide for Installation and Configuration of Single Node for Spark-oriented Distributed System

(VirtualBox + Ubuntu + Hadoop + Spark)

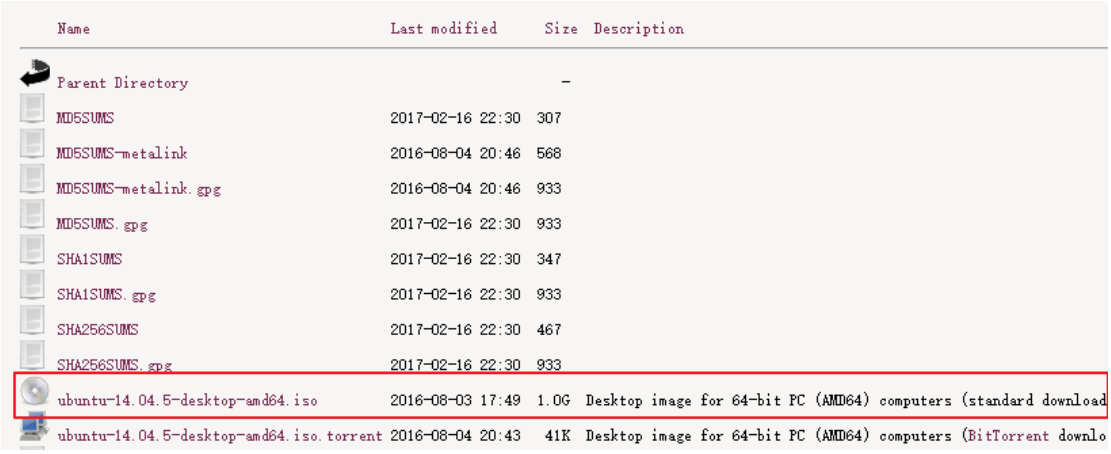
1. Preparation

Download the following installation packages:

1. VirtualBox 5.1.28: https://www.virtualbox.org/wiki/Downloads



2. Ubuntu 14.04 installation ISO file: http://releases.ubuntu.com/14.04/



Name	Last modified	Size	Description
Parent Directory		-	
MD5SUMS	2017-02-16 22:30	307	
MD5SUMS-metalink	2016-08-04 20:46	568	
MD5SUMS-metalink.gpg	2016-08-04 20:46	933	
MD5SUMS.gpg	2017-02-16 22:30	933	
SHA1SUMS	2017-02-16 22:30	347	
SHA1SUMS.gpg	2017-02-16 22:30	933	
SHA256SUMS	2017-02-16 22:30	467	
SHA256SUMS.gpg	2017-02-16 22:30	933	
ubuntu-14.04.5-desktop-amd64.iso	2016-08-03 17:49	1.0G	Desktop image for 64-bit PC (AMD64) computers (standard download
ubuntu-14.04.5-desktop-amd64.iso.torrent	2016-08-04 20:43	41K	Desktop image for 64-bit PC (AMD64) computers (BitTorrent downlo

3. Hadoop installation package: hadoop-2.7.4.tar.gz: http://www.apache.org/dyn/closer.cgi/hadoop/common/hadoop-2.7.4/hadoop-2.7.4.tar.gz
4. Java installation package: jdk-8u144-linux-x64.tar.gz: http://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html

Download the package in red frame:

Java SE Development Kit 8u144

You must accept the Oracle Binary Code License Agreement for Java SE to download this software.

Accept License Agreement Decline License Agreement

Product / File Description	File Size	Download
Linux ARM 32 Hard Float ABI	77.89 MB	jdk-8u144-linux-arm32-vfp-hflt.tar.gz
Linux ARM 64 Hard Float ABI	74.83 MB	jdk-8u144-linux-arm64-vfp-hflt.tar.gz
Linux x86	164.65 MB	jdk-8u144-linux-i586.rpm
Linux x86	179.44 MB	jdk-8u144-linux-i586.tar.gz
Linux x64	162.1 MB	jdk-8u144-linux-x64.rpm
Linux x64	176.92 MB	jdk-8u144-linux-x64.tar.gz
Mac OS X	226.6 MB	jdk-8u144-macosx-x64.dmg
Solaris SPARC 64-bit	139.87 MB	jdk-8u144-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit	99.18 MB	jdk-8u144-solaris-sparcv9.tar.gz
Solaris x64	140.51 MB	jdk-8u144-solaris-x64.tar.Z
Solaris x64	96.99 MB	jdk-8u144-solaris-x64.tar.gz
Windows x86	190.94 MB	jdk-8u144-windows-i586.exe
Windows x64	197.78 MB	jdk-8u144-windows-x64.exe

5. Spark installation package: spark-2.2.0-bin-without-hadoop.tgz: https://spark.apache.org/downloads.html

Download Apache Spark™

1. Choose a Spark release: 2.2.0 (Jul 11 2017)
2. Choose a package type: Pre-build with user-provided Apache Hadoop
3. Choose a download type: Direct Download
4. Download Spark: spark-2.2.0-bin-without-hadoop.tgz
5. Verify this release using the 2.2.0 signatures and checksums and project relea

Note: Starting version 2.0, Spark is built with Scala 2.11 by default. Scala 2.10 users build with Scala 2.10 support.

6. Scala Build Tool: sbt-0.13.16.tgz (the package will also install Scala) : http://www.scala-sbt.org/download.html

Windows

SBT-1.0.1.MSI

All platforms

SBT-1.0.1.ZIP SBT-1.0.1.TGZ

Previous releases

1.x

- sbt 1.0.0 (.zip) (.tgz) (.msi)

0.13

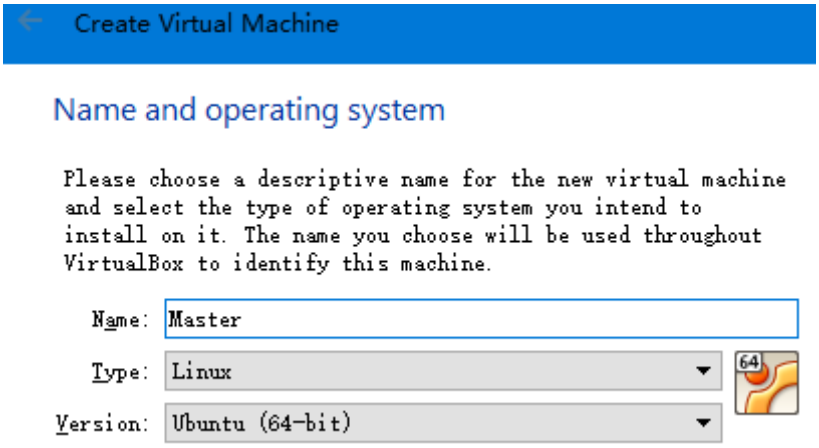
- sbt 0.13.16 (.zip) (.tgz) (.msi)
- sbt 0.13.15 (.zip) (.tgz) (.msi)

7. Scala IDE for Eclipse: http://scala-ide.org/

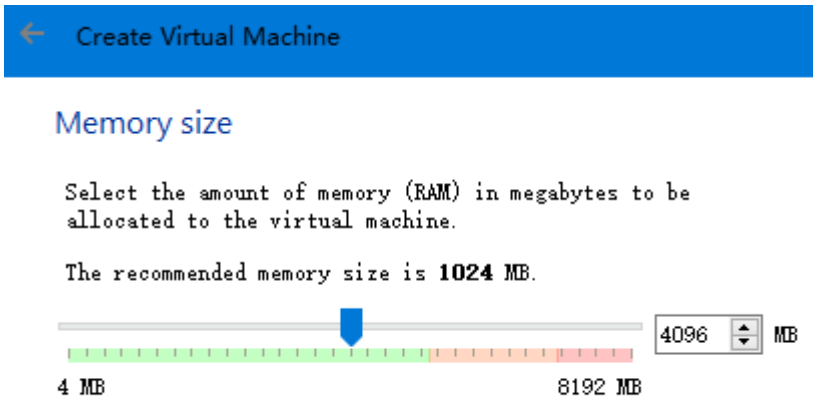
The installation package is “scala-SDK-4.6.1-vfinal-2.12-linux.gtk.x86_64.tar.gz”. You can extract the package in “home” directory.

2. Install Ubuntu in VirtualBox

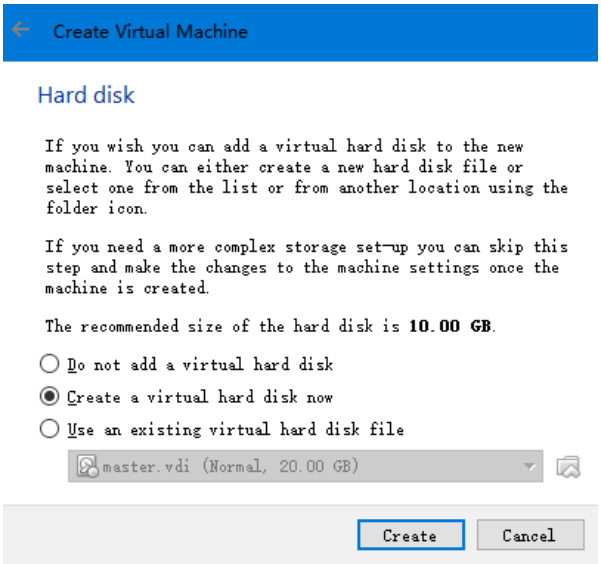
1. Click “New” button.
2. Input a name for new operating system and change type to Linux.



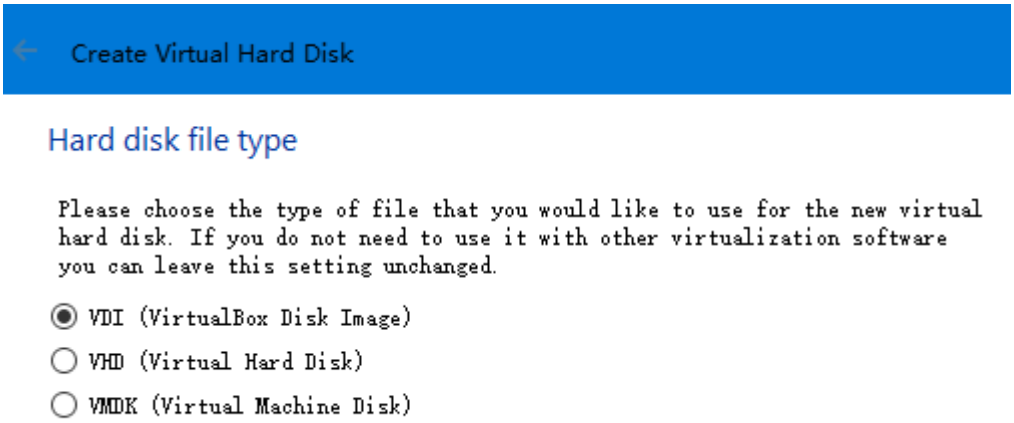
3. Select the size of RAM to be allocated for the new virtual machine, 4GB is recommended, 1GB at least :



4. For hard disk, select the default: “create a virtual disk now ”. Then, click “Create”:

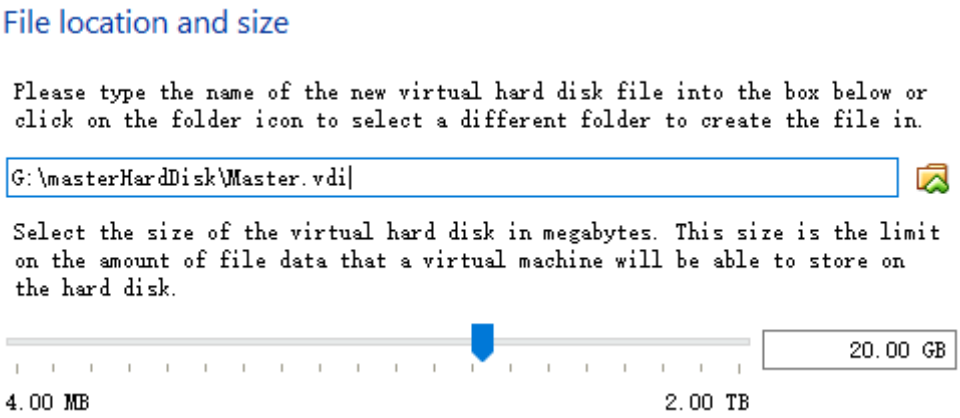


5. For hard disk file image, select the default: VDI:



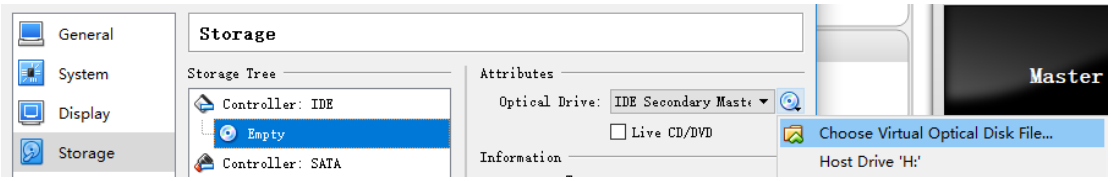
6. Then, select “Dynamically allocated” :

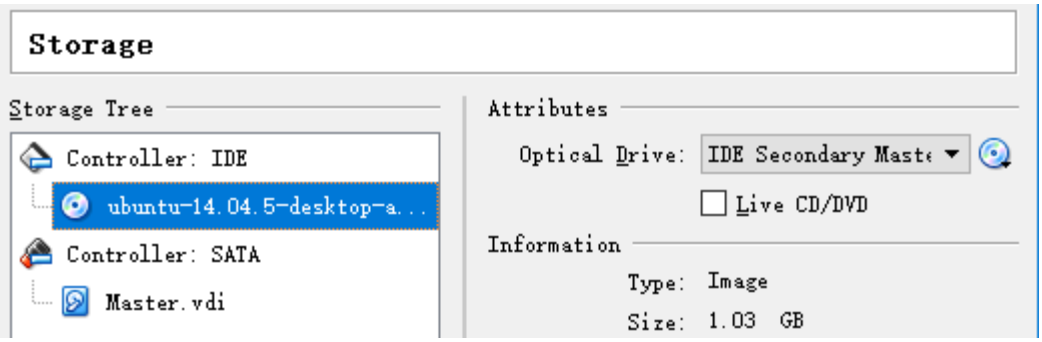
7. Select the path of the new virtual hard disk file and select the size of the virtual hard disk, 20GB is recommended:



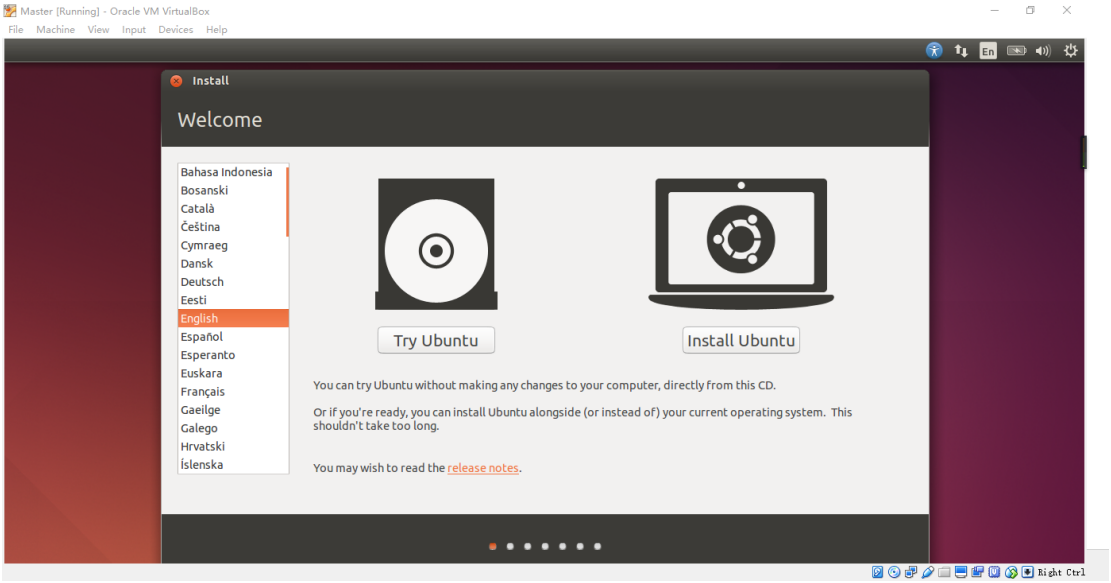
8. Click the “Create” button. Then, a virtual machine will be created.

9. Right click the Master. Select “settings” -> “storage”. In “Controller”, select “Empty”, click the CD icon in “Attributes”, select virtual optical disk file, select the Ubuntu installation ISO file.





10. Double click the Master virtual machine, and then click the “install Ubuntu”. The installation process of Ubuntu will start.



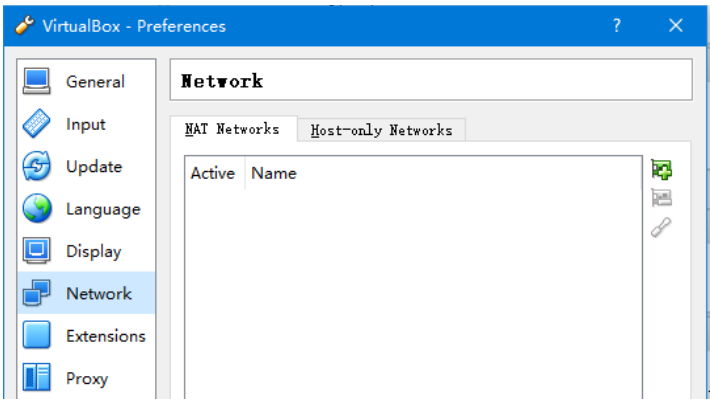
When the installation process prompts you to input the name of host and user, input the “Master” and “hadoop”. You should also set the password.

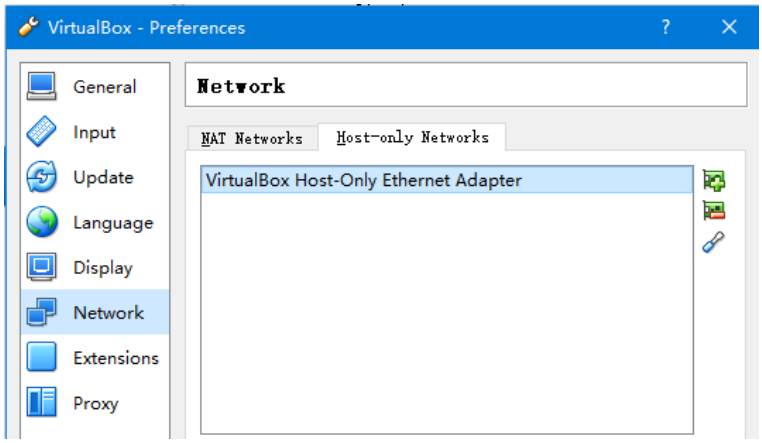
3. Network configuration

For network settings of VMs, different types have different features. The type of NAT can make VM be connected to Internet, but cannot make VMs be connected each other. The Host-only Adapter can make VM be connected to the Internet, as well as to other VMs. However, it cannot work in a network where some certification is needed usually. As many campus networks need certification, the network of the VM needs some special configuration to enable the VM to access not only the Internet but also the other VMs.

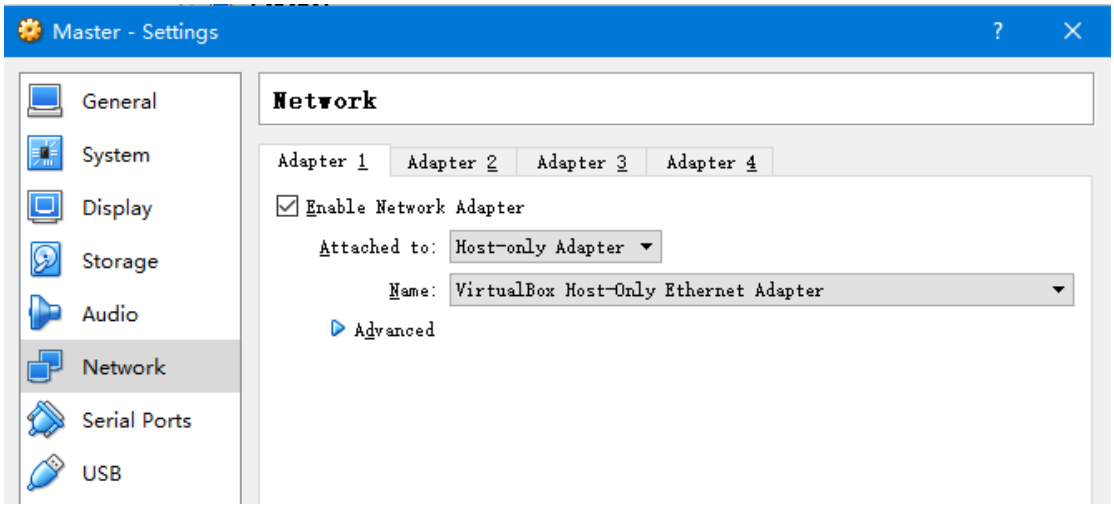
The following is a candidate scheme:

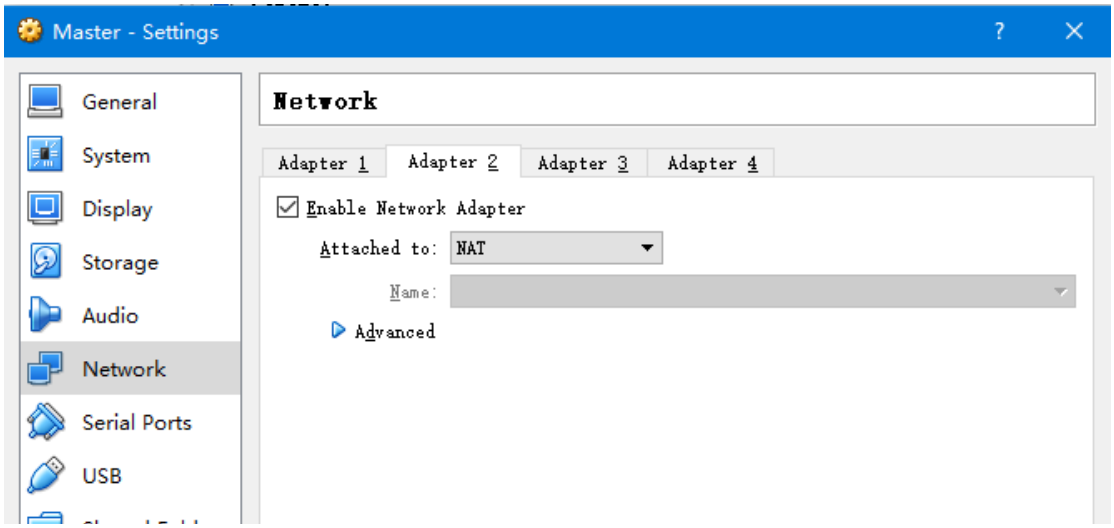
In Preferences of File of the menu of VirtualBox, keep the network settings as the default, like this:





For the settings of Master, right click the Master, select “Settings” -> “Network”. For the Adapter 1, select “Host-only Adapter”, it makes Master access other VMs. For the Adapter 2, select “NAT”, it makes Master access Internet.





Open the terminal in Ubuntu, input command: ifconfig, the Master contains two IP address:

```
hadoop@Master: ~
hadoop@Master:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:9f:14:46
          inet addr:192.168.56.101  Bcast:192.168.56.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe9f:1446/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:42 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1180 (1.1 KB)  TX bytes:7864 (7.8 KB)

eth1      Link encap:Ethernet  HWaddr 08:00:27:f8:7e:9e
          inet addr:10.0.3.15  Bcast:10.0.3.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fef8:7e9e/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:48 errors:0 dropped:0 overruns:0 frame:0
          TX packets:93 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7340 (7.3 KB)  TX bytes:12270 (12.2 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:173 errors:0 dropped:0 overruns:0 frame:0
```

Input command: route. Check the default network card. It should be the eth1, which adapts the NAT connection. Otherwise, change the default card to eth1.

```
hadoop@Master: ~
hadoop@Master:~$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         10.0.3.2        0.0.0.0         UG    0      0        0 eth1
10.0.3.0        *               255.255.255.0   U     1      0        0 eth1
192.168.56.0    *               255.255.255.0   U     1      0        0 eth0
hadoop@Master:~$
```

Input command: ping www.google.com –c 4, check if the VM can connect to the Internet.

4. Install Hadoop

4.1 Create user "hadoop"

In the installation of Ubuntu, user "hadoop" has been created, so it is not necessary to create it again.

4.2 Update apt

1. Input command: sudo apt-get update, which is to update repositories first. Otherwise, some software maybe cannot be downloaded.

2. Install vim: sudo apt-get install vim. Vim is a classic text editor in Ubuntu. You should learn how to use it.

4.3 Install SSH server and configure free password SSH login.

1. Install SSH server: sudo apt-get install openssh-server.
2. Input command to login the host: ssh localhost. It will create the directory "~/.ssh" ("~" represents the user's home folder, that is, "/home/username" directory. For example, if your user name is hadoop, then "~" represents "/home/hadoop/".)
3. Input command to exit from the host: exit.
4. Enter the directory: cd ~/.ssh/. In the Linux system.
5. Create rsa keys: ssh-keygen -t rsa. It will give you some hints. You can just click the Enter key.
6. Put the public rsa key into the authorized public key list file (~/.ssh/authorized_keys): cat ./id_rsa.pub >> ./authorized_keys. It makes "ssh localhost" work without password.
7. Input command: ssh localhost, to check if it is able to ssh the host without password.

4.4 Install Java environment

1. Create the Java installation directory: sudo mkdir /usr/local/java.
2. Extract the Java installation package to a specified directory: sudo tar -zxvf jdk-8u144-linux-x64.tar.gz –C /usr/local/java.
3. Set Java environment variable. Edit file "~/.bashrc": sudo vim ~/.bashrc by adding the following:

```
export JAVA_HOME=/usr/local/java/jdk1.8.0_144
export JRE_HOME=${JAVA_HOME}/jre
export CLASSPATH=.:$CLASSPATH:$JAVA_HOME/lib:$JRE_HOME/lib
export PATH=$PATH:$JAVA_HOME/bin:$JRE_HOME/bin
```

4. Make the new settings work: source ~/.bashrc.
5. Edit file "/etc/profile" in the same way. Add the content above and make them work.

6. Input command: java –version, to check if the environment variable works.

4.5 Install Hadoop

1. Extract the Hadoop package to a specified directory: sudo tar -zxvf hadoop-2.7.4.tar.gz -C /usr/local.
2. Enter the target directory: cd /usr/local/
3. Change the folder name to "hadoop": sudo mv ./hadoop-2.7.4/ ./hadoop. It makes the directory path simpler. Therefore, the environment variables would be simpler.
4. Change the owner of the folder: sudo chown -R hadoop:hadoop ./hadoop. It makes the user of "hadoop" have the permission to the Hadoop folder.
5. Enter Hadoop directory： cd /usr/local/hadoop
6. Check if Hadoop can be used： ./bin/hadoop version

4.6 Setting up a Single Node Cluster of Hadoop

1. Configuration files of Hadoop locate in /usr/local/hadoop/etc/hadoop/. Enter this directory.
2. Add Hadoop core configuration. Set hdfs (Hadoop file system) port as 9000. Edit core-site.xml as the following:

```
<configuration>
    <property>
        <name>hadoop.tmp.dir</name>
        <value>file:/usr/local/hadoop/tmp</value>
        <description>Abase for other temporary directories.</description>
    </property>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://localhost:9000</value>
    </property>
</configuration>
```

3. Add hdfs configuration information (namenode, datanode port and directory location) by editing hdfs-site.xml:

```
<configuration>
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
    <property>
        <name>dfs.namenode.name.dir</name>
```

```
        <value>file:/usr/local/hadoop/tmp/dfs/name</value>
    </property>
    <property>
        <name>dfs.datanode.data.dir</name>
        <value>file:/usr/local/hadoop/tmp/dfs/data</value>
    </property>
</configuration>
```

4. After the configuration, enter /usr/local/Hadoop directory, input command to format namenode: ./bin/hdfs namenode –format.
5. Start daemon of namenode and datanode : ./sbin/start-dfs.sh
6. If daemon starts successfully， you can visit web http://localhost:50070 to check the information of namenode and datanode.

5. Install Spark

1. Extract the Spark package to a specified directory: sudo tar -zxvf spark-2.2.0-bin-without-hadoop.tgz -C/usr/local.
2. Enter the directory: cd /usr/local.
3. Change the folder name to make it be used more easily later: sudo mv ./spark-2.2.0-bin-hadoop2.7/ ./spark.
4. Change the owner of the folder: sudo chown –R hadoop:hadoop ./spark.
5. Enter the directory: cd /usr/local/spark.
6. Set the environment of Spark. Spark has a template environment file. Creat a new one by copying the template: cp ./conf/spark-env.sh.template ./conf/spark-env.sh.
7. Edit file ./conf/spark-env.sh: vim ./conf/spark-env.sh, add the following to the end of file.

```
export SPARK_DIST_CLASSPATH=$(/usr/local/hadoop/bin/hadoop classpath)
```

8. Edit files of ~/.bashrc and /etc/profile by adding the following content to add Hadoop and Spark into the environment of system. Then, make them work (source ~/.bashrc and source /etc/profile):

```
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_CONF_DIR=${HADOOP_HOME}/etc/hadoop
export HADOOP_COMMON_LIB_NATIVE_DIR=${HADOOP_HOME}/lib/native
export OPTS="-Djava.library.path=${HADOOP_HOME}/lib"
export SPARK_HOME=/usr/local/spark
export PATH=${SPARK_HOME}/sbin:$PATH
export PATH=${SPARK_HOME}/bin:$PATH
export PATH=$HADOOP_HOME/bin:$HADOOP_HOME/sbin:$PATH
```

6. Install sbt 0.13.16（Scala build tool）

1. Extract the sbt package to a specified directory: sudo tar -zxvf sbt-0.13.16.tgz -C /usr/local/
2. Change the owner of the folder: sudo chown -R hadoop /usr/local/sbt. The "hadoop" is user name.
3. Enter the installation directory of sbt： cd /usr/local/sbt.
4. Create the sbt script in the directory /usr/local/sbt: vim ./sbt. Add the following. This script is used to launch sbt program.

```
#!/bin/bash
SBT_OPTS="-Xms512M -Xmx1536M -Xss1M -XX:+CMSClassUnloadingEnabled -XX:MaxPermSize=256M"
java $SBT_OPTS -jar /usr/local/sbt/bin/sbt-launch.jar "$@"
```

5. Add executable permission to ./sbt: chmod u+x ./sbt.
6. Input command: ./sbt sbtVersion. This command checks the version of sbt, and download some dependency files. It will take some time.

7. Pack Scala program by sbt

Create a Scala project "sparkapp" as a sample for understanding Scala and Spark according to the following :

1. Enter the main directory of user hadoop: cd ~.
2. Create the root directory of project: mkdir ./sparkapp.
3. Create the folder structure: mkdir -p ./sparkapp/src/main/scala. It is a standard structure of Scala project.
4. Create a new source code file named SimpleApp.scala in ./sparkapp/src/main/scala: vim ./sparkapp/src/main/scala/SimpleApp.scala, add following code:

```
/**
 * SimpleApp.scala
 * The program counts the number of lines that contain "a" or "b" in file /usr/local/spark/README
 */
import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._
import org.apache.spark.SparkConf

object SimpleApp {
  def main(args: Array[String]) {
    val logFile = "file:///usr/local/spark/README.md" // Should be some file on your system
```

```
    val conf = new SparkConf().setAppName("Simple Application") .setMaster("local[2]")
    val sc = new SparkContext(conf)
    val logData = sc.textFile(logFile, 2).cache()
    val numAs = logData.filter(line => line.contains("a")).count()
    val numBs = logData.filter(line => line.contains("b")).count()
    println("Lines with a: %s, Lines with b: %s".format(numAs, numBs))
  }
}
```

5. Create a new file named simple.sbt in ./sparkapp: vim ./sparkapp/simple.sbt. Add the following code. This is a description file for project dependency. It also sets the name and version of the project:

```
name := "Simple Project"
version := "1.0"
scalaVersion := "2.11.8"
libraryDependencies += "org.apache.spark" %% "spark-core" % "2.2.0"
```

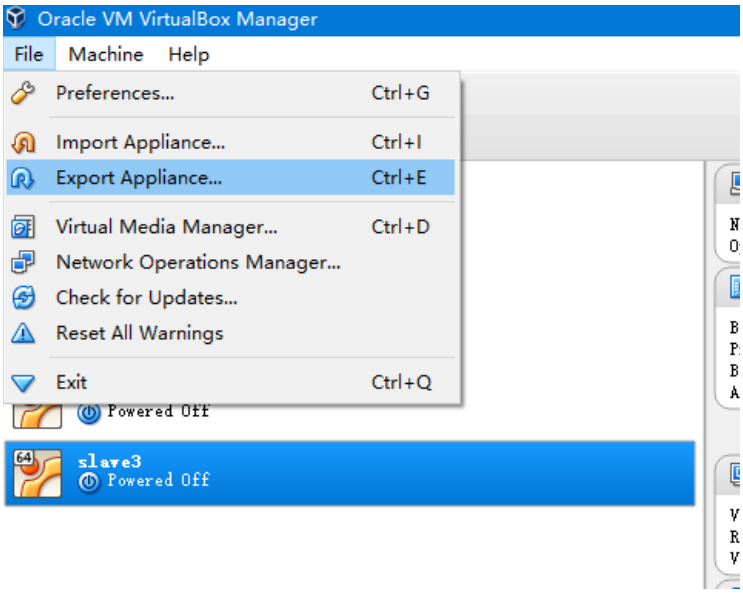
6. Check the structure of project by executing "cd ~/sparkapp", and "find .":

```
hadoop@Master:~/sparkapp$ find .
.
./src
./src/main
./src/main/scala
./src/main/scala/SimpleApp.scala
./simple.sbt
```

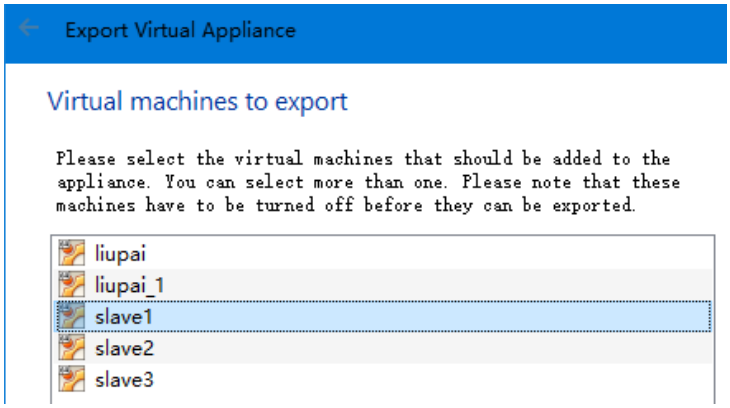
7. Build and pack the project in ./sparkapp: /usr/local/sbt/sbt package. the built package locates in ~/sparkapp/target/scala-2.11/simple-project_2.11-1.0.jar.

8 Export Appliance (to move VM to other PC)

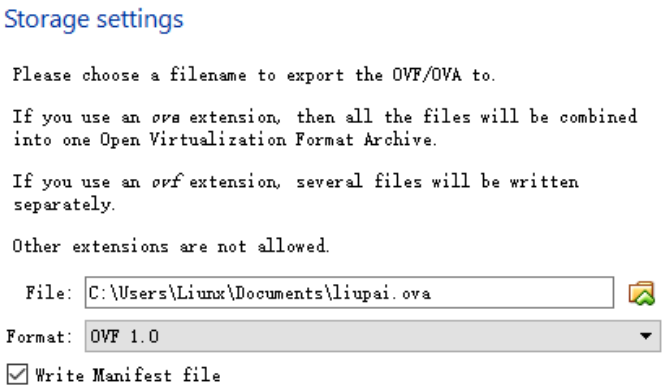
1. Select "Export Appliance" in "File" menu of VirtualBox.



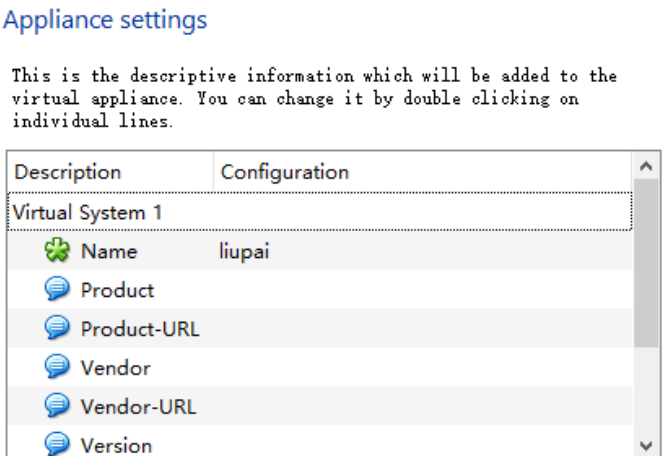
2. Select the VM that you want to export.



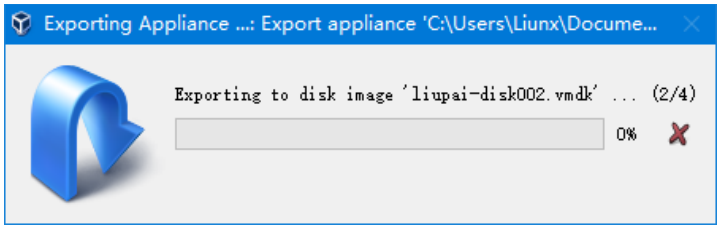
3. Configure the export settings. Select the storage path (the storage format should be OVF 1.0), and select “write manifest file”.



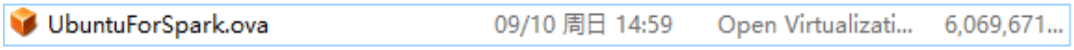
4. Next, you can add some descriptive information to the exported virtual machine, and others will see the information when they install it.



5. Click “Export” to begin the export operation.



6. An OVA file is generated after that. This is the exported virtual image:



Annex: Some Exceptions handling

1. Exception 1: datanode cannot start.

   You can try the following commands:

   1.1. ./sbin/stop-dfs.sh: stop the daemon process, shutdown Hadoop.

   1.2. rm -r ./tmp: delete the tmp directory, it would delete the data in hdfs. If your data is important, do backup in advance.

   1.3. ./bin/hdfs namenode –format: reformat the namenode.

   1.4. ./sbin/start-dfs.sh: restart Hadoop.

2. Exception 2:

   If you are prompted during formatting namenode by the error like: “Error: JAVA_HOME is not set and could not be found”. It is because the JAVA_HOME environment variable was not set up correctly. Please follow the aforementioned guide to set up the JAVA_HOME variable correctly. Otherwise, the subsequent processes cannot continue.